

π Tantum

alias *Tempus Tantum*

sistema di generazione orario scolastico

“Omnia, Lucili, aliena sunt, tempus tantum nostrum est.”
Seneca, *Epistulae morales ad Lucilium*, I, 1

Manuale generale

Parte I — per coordinatori d'orario

Una guida discorsiva all'uso quotidiano del sistema: preparazione dell'orario di inizio anno, gestione delle assenze, modifica in corso d'anno, creazione di corsi di recupero.

Pensata per chi usa π Tantum come strumento di lavoro, senza dover conoscerne i dettagli implementativi.

Giovanni Borghi

3 maggio 2026

Indice

1	Introduzione	3
1.1	Avvio rapido	4
1.2	Stato attuale del progetto	4
2	Installazione	5
2.1	Windows	5
2.2	Linux (Ubuntu, Debian, Fedora, Arch)	6
2.3	macOS (Intel + Apple Silicon)	7
2.4	Verifica installazione	7
2.5	Aggiornamento	8
2.6	Disinstallazione	8
3	Vincoli	9
3.1	I 5 stati / 4 kind	9
3.2	Matrici di disponibilità 5-stati	9
3.3	Vincoli logici (DNF)	9
3.3.1	Sintassi	9
3.3.2	Predicate atoms	10
3.3.3	Esempi pratici	10
3.3.4	Mapping CP-SAT	10
3.4	Vincoli HARD a toggle (per classe)	11
3.5	Conflict detection	11
3.6	Tag aule – etichette libere	11
3.7	Preferenze di giorno libero del docente	11
3.8	Preferenze di giorno libero della classe	12
4	Workflow di ottimizzazione	13
4.1	Schema delle fasi	13
4.2	Phase A: assegnazione (cattedre)	13
4.3	Phase B: scheduling	13
4.4	Metaeuristiche	13
4.5	Phase 4: assegnazione aule	14
4.6	Drag-and-drop con preview live	14
4.6.1	Room-follows-lesson	14
4.7	Slot picker matriciale (Monitor)	15
4.8	Punteggio di graduatoria e criterio Anzianita	15
5	Guida UI	16
5.1	Funzionalità trasversali	16
5.1.1	Query DSL e sort multi-livello	16
5.1.2	Multi-select	16
5.1.3	Excel/CSV import	16

5.1.4	Esportazione liste (xlsx / csv)	16
5.1.5	Viste salvate	17
5.1.6	Tag (aule e studenti)	17
5.1.7	Import bulk dedicato (/import)	17
5.1.8	Import / Export DB (Dashboard)	17
5.2	Tab per tab	18
6	Launcher	19
6.1	Windows: start.bat	19
6.2	Linux / macOS: start.sh + stop.sh	19
A	Riferimenti	20

1. Introduzione

Come usare questo manuale

La documentazione di π Tantum è suddivisa in due parti, per adattarsi a pubblici diversi:

- Il **Manuale generale** (Parte I, file `manual_generale.pdf`) è pensato per chi usa π Tantum come strumento di lavoro: dirigenti scolastici, coordinatori d'orario, docenti che devono modificare un vincolo o capire perché un orario è venuto in un certo modo. Tono discorsivo, casi d'uso concreti, niente jargon.
- Le **Appendici tecniche** (Parte II, file `appendici_tecniche.pdf`) sono per sviluppatori, amministratori IT e curiosi: architettura interna, modello dati, API REST, grammatica formale del DSL, dettaglio delle strategie di ottimizzazione e della diagnostica statistica.
- Il **Manuale completo** (file `manual_completo.pdf`) raccoglie entrambe le parti in un unico volume, utile per chi vuole stamparlo o averlo tutto a portata di mano.

Tutti e tre i PDF sono ricostruibili dai sorgenti `docs/*.tex` con lo script `docs/build_manual.sh` (o `.bat` su Windows).

Cos'è π Tantum

π Tantum (alias *Tempus Tantum*, codename interno: `timetable`) è un sistema di generazione e gestione dell'orario scolastico per un Liceo italiano. Il nome richiama il celebre incipit dell'epistola I di Seneca a Lucilio: “Omnia, Lucili, aliena sunt, tempus tantum nostrum est” (Seneca, *Epistulae morales ad Lucilium*, I, 1) – “tutto ci appartiene di altri, soltanto il tempo è nostro”. Il gioco grafico fra la lettera greca π e le due “T” di *Tempus Tantum* riassume l'idea: organizzare il tempo della scuola, dato che del tempo non si dispone se non gestendolo.

È composto di tre layer:

1. **Solver** (`experiments/`): pipeline CP-SAT (Google OR-Tools) con decomposizione spettrale per istanze grandi e cascata metaeuristica (LNS, SA, TS, ILS).
2. **Web UI** (`webui/`): backend FastAPI su SQLite + frontend SvelteKit. È il punto di accesso quotidiano: gestisce CRUD, ottimizzazione, visualizzazione, drag-and-drop, assenze e supplenze.
3. **Legacy** (`schedule/`): script monolitici da cui il solver è nato; mantenuti per riferimento storico.

Il manuale copre tutti e tre i layer ma si concentra sulla web UI, che ha assorbito quasi ogni funzionalità operativa.

1.1 Avvio rapido

Windows. Doppio click su `webui/start.bat`. Apre due finestre cmd: backend *uvicorn* su porta 8000, frontend *vite* su porta 5173.

Linux / macOS.

```
cd timetable/webui
./start.sh # background; URLs sulla console
./stop.sh # ferma backend + frontend
```

Nel browser, aprire `http://127.0.0.1:5173` e dalla **Dashboard** importare un profilo già calcolato (`small` per cominciare).

1.2 Stato attuale del progetto

Le funzionalità principali, al momento di questa redazione:

- 16 sezioni (tab) della UI, ~118 endpoint REST.
- ~30 tabelle SQLAlchemy con migrazioni idempotenti.
- 5 stati di vincoli (HARD/SOFT/PREFERITO/ENFORCED/ALLOWED).
- Vincoli logici DNF con DSL esteso (predicate atoms `aula:`, `materia:`, `classe:`, `gruppo:`, `docente:`).
- Conflict detection automatico nel tab *Vincoli*.
- Drag-and-drop con preview live (rosso/giallo/verde) e *room-follows-lesson*.
- Gestione assenze settimanali con assegnazione supplenti drag-drop.
- Tab *Monitor* con espansione lezione-per-lezione e slot picker matriciale.

2. Installazione

Guida cross-platform per installare ed eseguire piTantum su Windows, Linux e macOS. La versione *di riferimento* di questa guida è `docs/installation.md` nel repo (Markdown, sempre aggiornato); qui se ne riproduce il contenuto in forma compatta per il manuale PDF. In caso di divergenza, fa fede il file Markdown.

Architettura del lancio. Il progetto è composto da due processi:

- **Backend:** `uvicorn` che serve l'app FastAPI in `webui/backend` sulla porta 8000.
- **Frontend:** server di sviluppo `vite` che serve l'app SvelteKit in `webui/frontend` sulla porta 5173.

Uno script unico (`start.bat` su Windows, `start.sh` su Linux/macOS) li avvia entrambi. Al primo lancio crea la `venv` Python, installa i requirements, fa `npm install`; ai lanci successivi parte in pochi secondi.

2.1 Windows

Prerequisiti.

- Python ≥ 3.11 – installer ufficiale: <https://www.python.org/downloads/windows/>
- Node.js ≥ 20 LTS – installer ufficiale: <https://nodejs.org/en/download> (scegliere “LTS”).
- Git – installer ufficiale: <https://git-scm.com/download/win>.

Note importanti.

- Sull’installer Python, **spuntare** “Add python.exe to PATH”. Senza, lo script non trova il comando `python`.
- Riavviare il terminale dopo le installazioni: il PATH non si aggiorna nelle finestre già aperte.

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
webui\start.bat
```

Lo script apre due finestre cmd (“piTantum – backend” e “piTantum – frontend”). Aprire il browser su `http://localhost:5173`.

Stop. Chiudere le due finestre cmd, oppure `Ctrl+C` in ciascuna.

Troubleshooting Windows.

- “python non riconosciuto”: l’installer Python non è stato eseguito con “Add to PATH”. Reinstallare oppure aggiungere C:\Users\tu\AppData\Local\Programs\Python\Python311 al PATH.
- Porta 8000 occupata: `netstat -ano | findstr :8000 + taskkill /PID <pid> /F`.
- Antivirus blocca `start.bat`: aggiungere alle eccezioni di Windows Defender.

2.2 Linux (Ubuntu, Debian, Fedora, Arch)

Prerequisiti. Python ≥ 3.11 , Node ≥ 20 LTS, Git, build-essential.

Ubuntu / Debian.

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip nodejs npm \
    git build-essential
```

I repo apt di Ubuntu 22.04 / Debian 12 hanno Node *12-18*, troppo vecchio per SvelteKit 2 (richiede 20+). In quel caso usare **nvm**:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install --lts && nvm use --lts
```

Fedora.

```
sudo dnf install -y python3 python3-virtualenv nodejs npm git \
    gcc make
```

Arch / Manjaro.

```
sudo pacman -S python python-pip nodejs npm git base-devel
```

Clone + lancio.

```
git clone https://github.com/gborghi/timetable.git
cd timetable
chmod +x webui/start.sh webui/stop.sh
./webui/start.sh # background; log in webui/logs/
./webui/start.sh --foreground # log live in console (Ctrl+C ferma)
```

Stop.

```
./webui/stop.sh
```

Troubleshooting Linux.

- Porta in uso: `sudo lsof -i :8000 + kill <pid>` (eventualmente `kill -9 <pid>`).
- python3 mancante su distro minimal: usare l’eseguibile chiamato `python` (3.x).
- WSL2: `start.sh` funziona normalmente. Aprire il browser dal lato Windows su `http://localhost:5173`; WSL2 inoltra le porte automaticamente.

2.3 macOS (Intel + Apple Silicon)

Prerequisiti.

- Xcode Command Line Tools: `xcode-select -install`.
- Homebrew: vedere <https://brew.sh/>.
- Python ≥ 3.11 , Node ≥ 20 LTS, Git: tutti via Homebrew.

Installazione.

```
# Homebrew (se non presente)
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Toolchain
brew install python@3.12 node git

# Verifica
python3 --version # >= 3.11
node --version # >= v20
```

Note Apple Silicon.

- ortools dalla v9.7 spedisce wheel ARM64 nativi: Rosetta non serve.
- Se Python è installato da python.org invece che Homebrew, scegliere l'installer "macOS 64-bit universal2".

Clone + lancio + stop. Identico a Linux.

Troubleshooting macOS.

- Gatekeeper blocca `start.sh`: `xattr -d com.apple.quarantine webui/start.sh webui/stop.sh`.
- ortools senza wheel ARM (versioni vecchie): `pip install --upgrade ortools` dentro la venv del backend.
- brew non in PATH: `eval "$(/opt/homebrew/bin/brew shellenv)"` in `~/.zprofile`.

2.4 Verifica installazione

Dopo il primo lancio andato a buon fine:

```
python --version # >= 3.11 (Linux/macOS: spesso python3)
node --version # >= v20
npm --version # >= 10

# Backend health
curl http://127.0.0.1:8000/api/health
# atteso: {"status":"ok","name":"pitantum","version":"0.1.0"}

# Async layer (Section 2.5 P2)
curl http://127.0.0.1:8000/api/health/async
# atteso: {"status":"ok","async":true,"dialect":"sqlite"}

# Frontend
curl -I http://127.0.0.1:5173
# atteso: HTTP/1.1 200 OK
```


2.5 Aggiornamento

Quando si fa `git pull` per portare a casa nuovi commit:

```
cd /path/to/timetable
git pull origin main

# se sono cambiate dipendenze Python:
webui/backend/.venv/bin/pip install -r webui/backend/requirements.txt
# (Windows: webui\backend\.venv\Scripts\pip install ...)

# se sono cambiate dipendenze frontend:
cd webui/frontend && npm install && cd ../../

# se ci sono migration DB:
cd webui/backend && ../.venv/bin/alembic upgrade head && cd ../../
# (Windows: webui\backend\.venv\Scripts\alembic upgrade head)
```

In pratica `start.sh/start.bat` rilanciato dopo un `git pull` gestisce automaticamente `pip install` e `npm install` quando servono. Le migration DB sono coperte da `_apply_lightweight_migrations()` allo startup come `safety-net` (vedi capitolo *Modello dati*); per la via canonica usare `alembic upgrade head`.

Dopo un aggiornamento backend, fare un **hard reload** del browser (`Ctrl+Shift+R`) per pulire chunk JavaScript cached.

2.6 Disinstallazione

```
cd /path/to/timetable
./webui/stop.sh # Linux/macOS
# (Windows: chiudere le finestre cmd di start.bat)

rm -rf webui/backend/.venv # ~150 MB
rm -rf webui/frontend/node_modules # ~400 MB
rm -rf webui/data/timetable.db # DATI UTENTE (salvarli prima!)
```

Per rimuovere completamente, cancellare la cartella `timetable/`. Python, Node, Git restano installati nel sistema; rimuoverli dal gestore di pacchetti solo se non servono ad altri progetti.

3. Vincoli

3.1 I 5 stati / 4 kind

Il sistema usa una tassonomia uniforme di *5 stati* sia per le matrici di disponibilità (cella per cella) sia per le preferenze di aula. I vincoli logici disgiuntivi (DNF) hanno *4 kind* (ALLOWED non si applica qui).

Stato	Colore	Semantica
ALLOWED	verde chiaro	default (solo grid materia↔aula e docente↔aula); nessuna contribuzione all'obiettivo
SOFT	giallo	penalità positiva quando lo slot è usato (+ penalty aggiunto all'obiettivo, che è minimizzato)
HARD	rosso	il solver deve evitare questo slot, oppure il vincolo logico DEVE essere soddisfatto
PREFERITO	blu	bonus negativo quando lo slot è usato / il vincolo logico è soddisfatto (- bonus riduce l'obiettivo)
ENFORCED	verde scuro	il solver DEVE far accadere questo slot / soddisfare il vincolo con presence required

3.2 Matrici di disponibilità 5-stati

Il ciclo cliccando una cella è:

`free → soft → hard → preferred → enforced → free`

Le celle gialle e blu mostrano un input numerico per editare la penalità (positiva sul giallo, negativa sul blu, sign-clamp automatico al salvataggio).

Sincronizzazione free_day. Quando il docente ha `free_day` settato, tutte e 6 le ore di quel giorno appaiono HARD nella matrice automaticamente. Se l'utente edita la matrice manualmente e tutte le ore di un giorno diventano HARD, il backend deduce il `free_day`; viceversa cambiare il `free_day` sostituisce le auto-cells.

Mapping CP-SAT. In `optimization.py::_availability_constraints` ogni tabella si traduce in: `<entity>_hard` (set di tuple, forbidden assignment), `<entity>_soft` (dict tuple→penalty signed), `<entity>_enforced` (set di tuple, presence required nel solver).

3.3 Vincoli logici (DNF)

3.3.1 Sintassi

```

expr := or_expr
or_expr := and_expr ('OR' and_expr)*
and_expr := not_expr ('AND' not_expr)*
not_expr := ('NOT' | 'mai') not_expr | atom
atom := slot | predicate | '(' expr ')
slot := <giorno><ora>
predicate:= <kind> ':' <name> ('@' <giorno> <ora>)?
kind := aula | materia | classe | gruppo | docente | prof
giorno := lun|mar|mer|gio|ven|sab (anche lunedì|...|sabato,
        monday|...|saturday)
ora := 1..6 (ordinale, 1=08:00) | 8..13 (assoluta)

```

Sono accettati & (AND), | (OR), ! (NOT). mai è alias di NOT.

3.3.2 Predicate atoms

```

aula:LabFisica -- l'entita' usa l'aula a qualunque slot
aula:LabFisica@mar -- il martedì a qualunque ora
aula:LabFisica@mar4 -- martedì 4a ora (= 11:00)
materia:Religione@lun8 -- lezione di religione lunedì alle 8
classe:1A_Scientifico -- riferimento alla classe
gruppo:IRC -- riferimento al gruppo

```

I literal predicate vengono parsati e persistiti correttamente. Il solver li tratta otticamente per ora; l'integrazione strong con la fase classroom assignment è pianificata.

3.3.3 Esempi pratici

```

% Per docenti
lun8 AND lun9
(lun8 AND lun9) OR (mar8 AND mar9)
gio11 OR gio12 OR gio13
NOT (mer3 AND mer4)
mai aula:LabFisica@mar
mai materia:Religione@lun8

% Per classi
mer4 AND mer5 AND mer6
mai aula:Palestra@gio
aula:LabInformatica@mar OR aula:LabInformatica@gio

% Per aule
gio4 AND gio5 AND gio6
mai materia:Religione
gruppo:IRC OR gruppo:Alternativa

```

3.3.4 Mapping CP-SAT

Per ogni regola con DNF [clause1, clause2, ...]:

- HARD: il solver impone almeno una clausola fully active.
- SOFT: se nessuna clausola è fully active, paga +soft_penalty.
- PREFERITO: se almeno una clausola è fully active, ottiene +soft_penalty (negativo, quindi reduce dell'obiettivo).

- **ENFORCED**: come **HARD** ma con *presence required* – almeno uno degli slot della DNF deve coincidere con un’ora di lezione attiva per quell’entità.

3.4 Vincoli **HARD** a toggle (per classe)

In `school_classes` ci sono 7 boolean strutturali: `hard_entry_at_8`, `hard_exit_after_12`, `hard_no_holes`, `hard_dual_math`, `hard_dual_italian`, `hard_motorie_pairs`, `hard_max_6_per_day`. Il soft `soft_minimize_sixth_weight` regola il peso della 6a ora.

3.5 Conflict detection

Il tab *Vincoli* ha un bottone "Cerca conflitti" (GET `/api/monitor/conflicts`) che segnala:

- **matrix_hard_enforced** – stessa cella marcata sia **HARD** sia **ENFORCED**.
- **enforced_on_free_day** – cella **ENFORCED** del docente nel suo `free_day`.
- **logical_unsatisfiable** – vincolo logico **HARD/ENFORCED** la cui DNF non è soddisfacibile dato l’insieme degli **HARD** slot dell’owner.

3.6 Tag aule – etichette libere

Per esprimere “questa aula ha il proiettore”, “questa aula è adatta alla matematica perché ha la lavagna grande”, “questa aula è una sala riunioni”, il sistema offre i **tag delle aule**: etichette libere che il coordinatore attacca dalla scheda Aule. Ogni aula può avere quanti tag vuoi; ogni tag può essere riutilizzato su più aule.

I tag sono utilissimi nei vincoli: invece di scrivere “solo queste tre aule precise” (lista che andrebbe aggiornata ogni volta che le aule cambiano), scrivi “aule taggate matematica” e il vincolo segue automaticamente le etichettature correnti.

Esempio: vogliamo che la matematica si tenga solo in aule taggate `matematica`. Una volta etichettate le aule giuste dalla scheda Aule, il vincolo nel DSL generico è:

```
forall l in lessons where l.subject == Matematica:
    "matematica" in l.classroom.tags
```

Il sistema ha anche una serie di tag automatici prodotti dalla generazione mock (per esempio le aule home class ricevono il tag dell’indirizzo della classe assegnata). Per dettagli vedi il modulo `webui/backend/mock_classrooms.py`.

3.7 Preferenze di giorno libero del docente

I docenti hanno diritto contrattuale a un certo numero di giorni liberi a settimana (tipicamente uno per il CCNL italiano). π Tantum gestisce questo aspetto in modo flessibile attraverso tre slot di preferenza ordinati:

- **1ª preferenza** – il giorno libero ideale del docente (es. “sabato”).
- **2ª preferenza** – il giorno di ripiego se il primo non è possibile (es. “mercoledì”).
- **3ª preferenza** – l’ultima alternativa (es. “lunedì”).

Per ciascuna preferenza il docente (o il coordinatore) sceglie se è **HARD** (assoluta) o **SOFT** con un peso. Più un campo `required_free_days_count` indica *quanti* giorni liberi servono in totale (di default 1 per CCNL).

Esempio: un docente con preferenze [sabato **HARD**, mercoledì **SOFT** peso 50] e `count=1` vorrà sicuramente sabato libero, e in secondo ordine il mercoledì se il sabato non è fattibile.

3.8 Preferenze di giorno libero della classe

Anche le classi possono avere giorni preferiti di non-attività. La struttura è analoga a quella dei docenti: tre slot ordinati con flag HARD/SOFT, più un **required_free_days_count**.

In aggiunta le classi hanno il campo **max_hours_per_day** (default 5) che impone un tetto HARD al numero di ore in un giorno. Riducendolo (a 4 per esempio) si ottengono giornate più leggere ma se il monte ore della classe è alto bisognerà estendere il calendario settimanale al sabato.

4. Workflow di ottimizzazione

4.1 Schema delle fasi

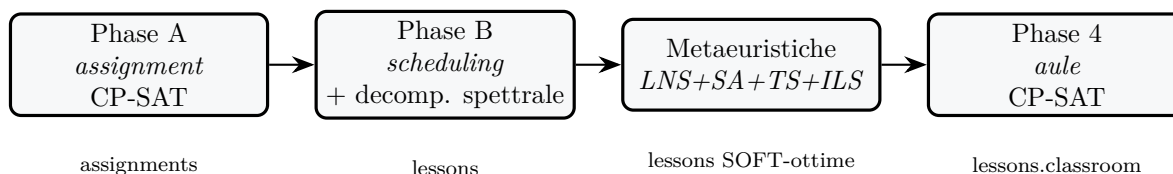


Figura 4.1: Pipeline a 4 fasi.

4.2 Phase A: assegnazione (cattedre)

Modulo. `experiments/cpsat_v2_assignment.py`.

Lancio. `POST /api/optimize/assignment` con `{time_limit_s, workers, log}`.

Input. L'insieme di `(class, subject, hours_per_week)` letto da `class_subjects` e/o `curriculum_subjects` + le abilitazioni docente in `teacher_subjects` + la classe-di-concorso preference in `subject_group_weights`.

Output. La tabella `assignments` (`teacher x class x subject` con `hours`). Problema di covering: ogni `(class, subject)` riceve un docente abilitato, rispettando `max-hours` per docente, classi-di-concorso e `teacher_compatible_classes`.

4.3 Phase B: scheduling

Modulo. `experiments/cpsat_v2_timetable.py` + `experiments/decomposition_spectral_v2.py`.

Decomposizione spettrale. Per istanze grandi, il problema CP-SAT integrale è intrattabile. La pipeline calcola un grafo di co-occupazione fra docenti, ne fa un'embedding spettrale via Laplaciano normalizzato e usa K-means su quell'embedding per partizionare in k cluster *loosely-coupled*. Ogni cluster diventa una sub-istanza CP-SAT indipendente; un secondo step "bridges" risolve solo i bordi; un terzo step "ricucitura" fa full-CPSAT con tutto il problema partendo dalla soluzione stitched.

4.4 Metaeuristiche

Modulo. `experiments/metaheuristics.py`. Cascata di:

- **LNS** – destroy-and-repair su finestre (`teacher`, `day`) o (`class`, `day`); freeze del 60-70% delle lezioni e CP-SAT-resolve del resto.
- **SA** (Simulated Annealing) – single-swap accettati con Metropolis a T decrescente con fattore α .
- **TS** (Tabu Search) – best-improvement local search con tabu list (default size 80).
- **ILS** (Iterated Local Search) – alterna n cicli di TS con *kick* perturbativi (random 4-opt fra giorni).

Tutti contribuiscono al SOFT score: holes per teacher, isolated 1- and 5-hour days, weekly distribution, dual-hour pairs, no 6th hour, banded preferences su materie. Il SOFT include la contribuzione delle matrici di disponibilità SOFT/PREFERITO e dei logical SOFT/PREFERITO.

4.5 Phase 4: assegnazione aule

Modulo. `experiments/classroom_assignment.py`. Input: la tabella `lessons` (già temporalmente fissata) + le classroom rules (`kind`, `capacity`, `multi_class`, preferenze materia↔aula, classe↔aula, docente↔aula). Output: `lessons.classroom_name` popolato.

4.6 Drag-and-drop con preview live

Trascinando una lezione su un altro slot la UI fa due chiamate:

1. POST `/api/schedule/move-preview` – il backend simula la mossa per tutte le 36 (`day`, `hour`) e per ognuna restituisce:
 - `ok` con `delta_soft <= 0` – mossa accettabile (verde se `delta < 0`, verde chiaro se `delta = 0`).
 - `soft_worse` – mossa accettabile ma peggiora il SOFT (giallo).
 - `hard_violation` – vincolo HARD violato (rosso).
 - `noop` – slot di origine.
2. L'utente droppa, la UI invia PUT `/api/schedule/move-lesson` che valida e persiste.

4.6.1 Room-follows-lesson

Il move-preview NON considera l'aula nel calcolo HARD. Se la nuova posizione è compatibile con docente / classe ma l'aula attuale è già occupata o HARD-unavailable in destination, il backend:

1. Sposta la lezione.
2. Lascia l'aula libera sul nuovo slot solo se non c'è conflitto.
3. Se c'è conflitto, rimuove l'aula dalla lezione spostata e risponde con `room_cleared=true`, `cleared_room=<nome>`.
4. La UI apre un Modal che spiega cosa è successo e chiede di scegliere una nuova aula dal dropdown (verde libera / rosso occupata).

4.7 Slot picker matriciale (Monitor)

Nel tab *Monitor*, espandendo una cattedra si vedono le singole lezioni; cliccando il bottone *Giorno/Ora* si apre un modal con una matrice 6x6 dove ogni cella è colorata in base alla disponibilità:

- verde: free (docente e classe liberi)
- rosso: HARD (docente o classe già impegnati)
- ambra: aula occupata da un'altra lezione
- azzurro: slot attuale

Click su una cella libera fa un dry-run via PUT `/api/monitor/event/{aid}/lesson/{lid}`. Se ci sono conflitti, apre un secondo Modal con 3 opzioni: **Annulla**, **Disassegna le lezioni in conflitto**, **Disassegna e ottimizza dopo**.

4.8 Punteggio di graduatoria e criterio Anzianita

Ciascun docente nell'anagrafica ha un campo **graduatoria_score** (numerico). Il valore esprime il punteggio di graduatoria utile a discriminare i docenti quando si tratta di assegnare cattedre o supplenze: più alto = maggiore precedenza.

Il campo è usato in due punti:

1. **Modulo supplenze**: quando il sistema cerca un candidato per coprire un'assenza, ordina la rosa per **graduatoria_score** decrescente.
2. **Phase A, criterio “Anzianita”**: il DSL della funzione obiettivo della Phase A include un preset **Anzianita** che, oltre ai criteri standard di bilancio (capacità non utilizzata, n. di indirizzi, ecc.), privilegia le assegnazioni che fanno coincidere docenti con alto **graduatoria_score** con classi del triennio finale (a parità di altre condizioni).

Il preset è selezionabile dalla card **Phase A (assegnazione)** del tab Workflow, dropdown “Criterio”. Il DSL completo della funzione obiettivo è visibile nel modal “Mostra DSL attivo” della stessa card.

5. Guida UI

5.1 Funzionalità trasversali

5.1.1 Query DSL e sort multi-livello

Ogni pagina lista ha:

- barra Query (DSL: =, !=, <, <=, >, >=, contains, startswith, endswith, in [...], logica AND / OR, parentesi);
- sort multi-livello (max 4 livelli);
- pannello Help con la lista dei campi.

5.1.2 Multi-select

Sulle pagine Docenti / Classi / Aule la lista supporta:

- click semplice: single-select (replace);
- Ctrl+click: toggle;
- Shift+click: range;
- bottone "Vincolo collettivo" che apre il *BulkApplyModal*.

5.1.3 Excel/CSV import

Ogni pagina lista ha "Importa Excel/CSV" + "Template". Lo stesso endpoint POST `/api/import/{entity}` gestisce le 7 entità (teachers, subjects, classes, classrooms, curricula, students, groups). Modalità: **upsert** (default), **append**, **replace**. Header alias italiani / inglesi accettati.

5.1.4 Esportazione liste (xlsx / csv)

Sopra ogni tabella, accanto a "Reset query" / "Reset sort", trovi 3 bottoni di esportazione:

- **xlsx**: la *vista corrente* (solo le righe filtrate dalla query DSL, nell'ordine determinato dal sort multi-livello) come file `.xlsx` con header colorato e auto-fit colonne.
- **csv**: stesso contenuto, formato CSV UTF-8 con BOM (compatibile con Excel italiano).
- **tutto**: ignora query e sort, esporta tutte le righe dell'entità.

Filename: `<entita>_<YYYYMMDD>_<HHMMSS>.<ext>`. I parametri `?format=xlsx|csv` sono accettati da tutti gli endpoint di lista, quindi gli stessi export sono disponibili anche via curl.

5.1.5 Viste salvate

Sopra ogni tabella, accanto ai bottoni di Reset e Export, trovi un dropdown "Viste salvate..." e i bottoni "Salva vista" / "x":

- **Salva vista:** chiede un nome e memorizza la query DSL + il sort multi-livello come `SavedView(entity, name, dsl_query, sort_levels)`. La vista è globale.
- **Dropdown "Viste salvate...":** applica una vista (sostituisce query + sort + ricarica la lista).
- **x:** elimina la vista correntemente selezionata.

Endpoint: `/api/saved-views?entity=<entity>` (GET / POST / PUT {id} / DELETE {id}). Tabella: `saved_views`.

5.1.6 Tag (aule e studenti)

Sistema many-to-many parallelo per aule e studenti.

Tag aule: tabella `classroom_tags` + join `classroom_tag_assignments`. UI: multi-select autocomplete nella scheda aula, bottone "Gestisci tag" per CRUD globale, colonna "Tag" con pill colorate (un colore stabile per nome). Il generatore mock tagga automaticamente le aule per kind (`lab_fisica` → [`lab`, `fisica`, `scienze`], `palestra` → [`palestra`, `motorie`], ecc.) e propaga il curriculum dalla home class (`3B_scientifico` → `tag_scientifico`).

DSL: "matematica" in `l.classroom.tags` (vincoli) o `has_tag(matematica)` (query lista). I tag sconosciuti vengono creati al volo al salvataggio.

Tag studenti: tabella `student_tags` + join `student_tag_assignments`. Casi d'uso: BES, DSA, `debito_matematica_4`, `pcto_ditta_x`, `studente_atleta`. UI analoga a quella delle aule. Pannello "Precompila da tag" nel modal *Gruppi* che aggiunge in massa gli studenti che matchano `any_of` o `all_of` su una lista di tag.

Importante: i tag NON sostituiscono i gruppi. I gruppi restano l'unità operativa di scheduling, i tag sono metadato passivo per filtrare/raggruppare.

5.1.7 Import bulk dedicato (/import)

Pagina dedicata raggiungibile dalla nav. Layout a due colonne: configurazione (entità, modalità, "Scarica template") + file da importare (drop area + selettore file). Il template xlsx ha 3 fogli:

- **Istruzioni:** tabella **Campo** | Hint con descrizione di ogni colonna + note generali (sinonimi italiani, righe vuote ignorate, etc.);
- **Esempi:** una riga di esempio pre-popolata;
- **Dati:** foglio vuoto da compilare. L'importer legge per default questo foglio.

Campi importabili già integrati con le feature recenti: `tags` su `students` e `classrooms`, `graduatoria_score` / `required_free_days_count` / `preferred_free_days_json` su `teachers`, e `max_hours_per_day` su `classes`.

5.1.8 Import / Export DB (Dashboard)

Card prominente nella prima riga della Dashboard. Funzioni:

- **Esporta DB completo:** scarica un `.zip` contenente `database.db` (raw SQLite), `tables/<t>.csv` per ogni tabella e `metadata.json` (schema_version + sha256).

- **Solo schema:** omette i dati di scheduling – utile come template per inizializzare un'altra scuola.
- **Import:** selettore file .zip + conferma esplicita. Una copia `timetable.db.pre_import_backup` viene salvata prima dello swap.
- **Snapshot:** timestampati in `webui/data/snapshots/`, lista con bottoni Ripristina / Elimina.

Endpoint: `/api/dashboard/export-db`, `/api/dashboard/import-db`, `/api/dashboard/snapshot/{create}`, `/api/dashboard/snapshot/{delete}`, `/api/dashboard/snapshot/{f}`.

5.2 Tab per tab

Dashboard

Importa profilo (5 size) con 3 checkbox per i pool (curricula, aule, studenti); genera scuola di test; stato corrente con 7 card-numero; *RunLogPanel* streaming.

Docenti, Classi, Indirizzi, Studenti, Gruppi, Materie, Aule, Compresenze

CRUD entità con modal di edit. Vedere `ui_guide.md` per i campi specifici di ognuna.

Cattedre

Layout a card: una card per classe, lista materie+docente+ore. Bottone "cambia": apre modal con dropdown filtrato per materia, ogni opzione mostra `Cognome Nome - assigned/max [SFORA: +Xh] / [pieno]`. Bottone "Warnings cattedre": pannello con docenti SFORA / sotto / vuoti / ok.

Orario

4 viste (per classe, per docente, per aula, per slot), ognuna con toggle matrice/lista. Drag-and-drop con preview live, dropdown aule colorate (verde libera / rosso occupata).

Assenze e supplenze

Vista settimanale 6×6. Click su intestazione giorno: modal gestione assenze. Click su cella rossa: modal con classi scoperte (sinistra) + docenti disponibili (destra), drag-drop per assegnare supplenti.

Monitor

Lista *eventi* (uno per Assignment). Espansione lezione-per-lezione con bottone `Giorno/Ora` che apre slot picker matriciale (vedere §5.6). In cima alla pagina un *segmented control* con quattro tab per filtrare velocemente: **Tutti**, **Incompleti** (eventi con `n_assigned_hours < n_expected_hours`), **Senza orario** (eventi senza alcuna lezione piazzata) e **Lockati** (eventi con almeno una lezione marcata `lesson.locked == true`). Per ogni riga, le azioni disponibili sono *Dissocia*, *Blocca/Sblocca* e *Piazza* (eseguito singolarmente o in massa via multi-selezione + toolbar bulk).

Vincoli

Lista flat di tutti i vincoli editabili con pill colorate per livello. Bottone "Cerca conflitti": pannello con conflitti rilevati.

Workflow

Pulsanti per lanciare le 4 fasi separatamente o "Pipeline completa". SSE log streaming.

6. Launcher

6.1 Windows: `start.bat`

Apri due finestre cmd separate. Pre-flight:

- `npm` reachable (forza `nodejs` nel PATH);
- `venv` esiste in `backend/.venv`;
- `backend/main.py` e `frontend/package.json` presenti;
- se `node_modules` manca, lancia `npm install`;
- avvisa se 8000 / 5173 sono già occupate.

I path sono relativi a `%~dp0`, quindi lo script funziona ovunque venga messo.

6.2 Linux / macOS: `start.sh` + `stop.sh`

Default background: log in `webui/logs/`, PID in `webui/logs/pids`. Flag `-f` / `-foreground` per Ctrl+C cleanup. Auto-crea `venv` e fa `npm install` se mancano. Hint per `apt`, `dnf`, `pacman`, `brew` se `python3` / `npm` non sono nel PATH.

`stop.sh` legge `logs/pids`, manda SIGTERM con fallback SIGKILL dopo 3s, killa anche i child via `pkill -P` e ha un fallback per ammazzare qualunque processo ancora in ascolto su 8000 / 5173 via `lsof`.

A. Riferimenti

Per i dettagli tecnici (architettura, modello dati, API REST, DSL formale, strategie di ottimizzazione, diagnostica statistica, estendere il sistema) vedere il volume separato **Appendici tecniche** (file `appendici_tecniche.pdf`). Per avere entrambi i volumi in un unico documento, vedere `manual_completo.pdf`.

Glossario di tutti i termini tecnici: file `docs/glossario.md` nel repository.